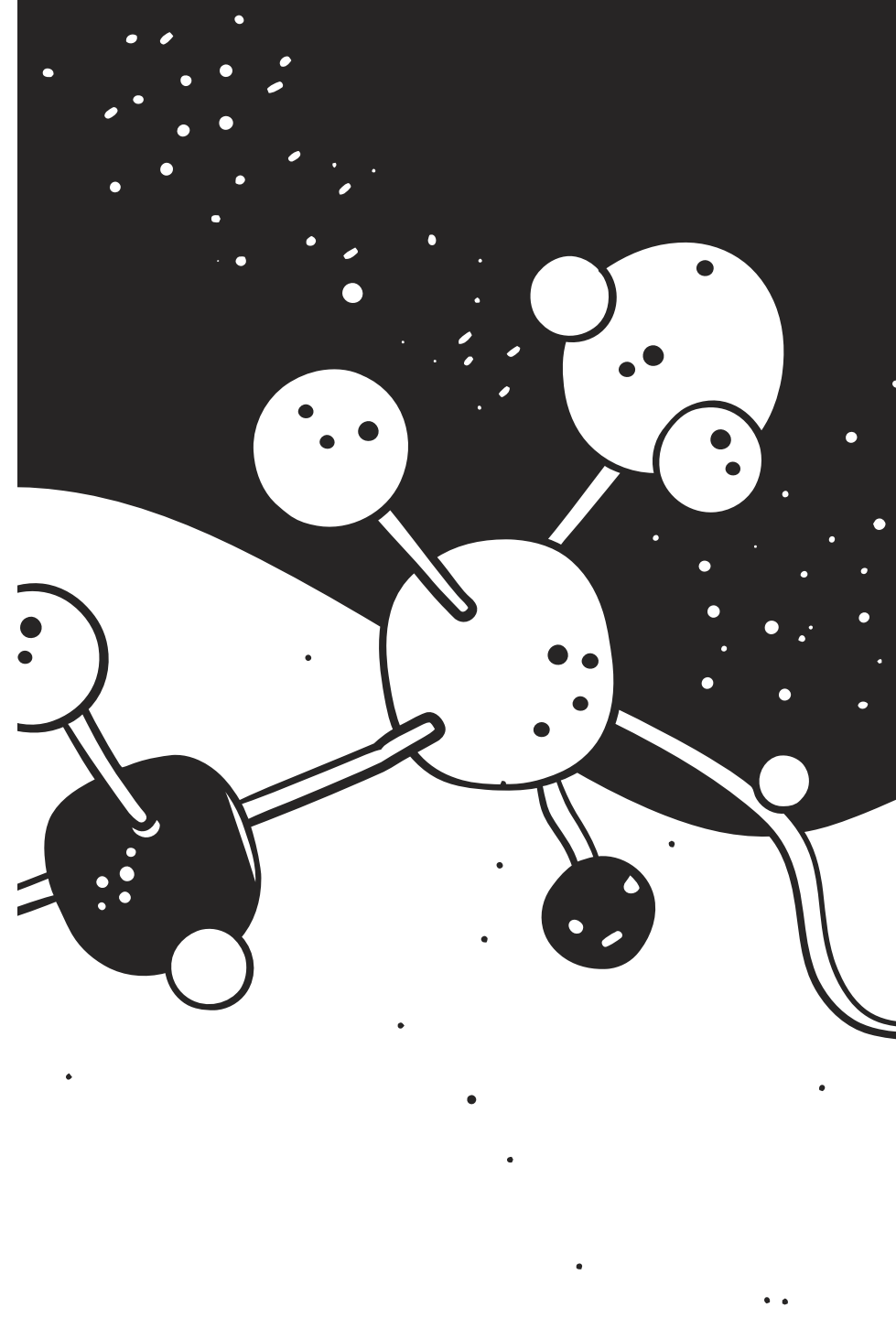


Teoria R2 — Decisions, arrays, funcions i regles de domini

Preparació per a R2M3: condicions, arrays, funcions pròpies i fitxer de regles en PHP.

CICLE FORMATIU · PHP



Índex de continguts

01

Regles de domini

Del llenguatge natural al codi

02

Arrays de domini

Estructures amb significat real

03

Funcions útils

Nom clar, entrada i responsabilitat

04

Fitxer de regles

require_once i separació de codi

05

Casos de prova

Demostrant que la regla funciona



Què és una regla de domini?

Una **regla de domini** és una decisió del producte expressada en codi. No és lògica tècnica arbitrària: és un requisit real del negoci o projecte que condiciona el comportament del sistema de manera **visible i verificable**.



Llenguatge natural

«Només es pot confirmar una reserva si està en estat pendent i no té conflicte»



Codi PHP

```
$estat === 'pendent' &&  
!$teConflicte
```



Resultat

El sistema actua diferent segons si la condició es compleix o no

De la regla natural a la condició

La regla en paraules

«Una reserva es pot confirmar si el seu estat és *pendent* i no hi ha cap conflicte d'horari.»

Qualsevol membre de l'equip ha d'entendre la regla sense llegir codi. Si no s'entén en paraules, no s'entendrà en codi.

Traducció a PHP

```
function potConfirmarReserva(  
    string $estat,  
    bool $teConflicte  
): bool {  
    return $estat === 'pendent'  
        && !$teConflicte;  
}
```

Arrays de domini: informació amb significat

Un array de domini **no és una llista qualsevol**. Representa informació real del projecte: catàleg, prioritats, estats possibles, tipus de recursos o etiquetes.

Exemple: prioritats

```
$prioritats = [  
  'baixa' => 1,  
  'normal' => 2,  
  'alta' => 3,  
];
```

Les claus són llegibles per humans; els valors permeten comparar i ordenar.

Altres usos habituals

Catàleg

Productes, serveis o recursos disponibles

Estats

pendent, confirmat, cancel·lat...

Etiquetes

Tipus d'incidència, categories, rols

Una funció útil: nom, entrada i responsabilitat



Nom clar

El nom ha de dir *què fa* la funció.
`potConfirmarReserva()` és millor que
`check()` o `fn1()`.



Dades d'entrada

Rep paràmetres tipats: `string $estat`,
`bool $teConflicte`. Mai literals fixos
interns.



Responsabilitat única

Retorna una sola cosa: `bool`. No mostra
res per pantalla ni modifica variables
globals.

 La lògica ha d'estar connectada a dades reals del flux, **mai a literals fixos** escrits dins la funció.

El fitxer `regles.php` i `require_once`

src/regles.php

```
<?php
// Regles de domini del projecte

function potConfirmarReserva(
    string $estat,
    bool $teConflicte
): bool {
    return $estat === 'pendent'
        && !$teConflicte;
}
```

Un fitxer separat amb funcions o regles del projecte. Encara no és MVC ni arquitectura completa, però ja és una separació real de responsabilitats.

Flux principal

```
require_once __DIR__
    . '/../src/regles.php';

if (potConfirmarReserva(
    $estat,
    $teConflicte
)) {
    $missatge =
        'Reserva confirmable';
} else {
    $missatge =
        'Cal revisar la reserva';
}
```

- 📄 `require_once` garanteix que el fitxer es carrega una sola vegada i llança error si no existeix.

Dos casos de prova: demostrant que la regla funciona

La manera mínima de verificar una regla és provar-la amb **dos casos amb resultat diferent**.

✓ Cas 1 — Confirmable

```
$estat    = 'pendent';  
$teConflicte = false;  
  
// Resultat: true  
// → 'Reserva confirmable'
```

✗ Cas 2 — No confirmable

```
$estat    = 'pendent';  
$teConflicte = true;  
  
// Resultat: false  
// → 'Cal revisar la reserva'
```

🕒 Si els dos casos donen resultats oposats i esperats, la funció implementa la regla correctament.



Errors habituals

→ **Condicció sense efecte visible**

La condició existeix però no canvia res en la resposta de l'usuari. Tota regla ha de tenir conseqüència observable.

→ **Literals fixos dins la funció**

Escriure valors hardcodejats dins la funció trenca la reutilització. Passa les dades com a paràmetres.

→ **Array sense connexió al flux**

Definir un array però no usar-lo en cap decisió ni sortida no aporta valor real al projecte.

→ **Un sol cas de prova**

Provar només el camí feliç no demostra que la regla funciona. Cal sempre el cas que retorna fals o error.

Preguntes per al teu projecte

Reflexiona sobre el teu projecte R2 i respon:

1

Regla real

Quina regla real del teu projecte pots escriure en llenguatge natural?

2

Dada del flux

Quina dada del formulari o del flux alimenta eixa regla?

3

Array de domini

Quin array representa informació real del teu domini?

4

La funció

Què rep i què retorna la funció que implementa la regla?

 Si pots respondre les quatre preguntes, tens tot el necessari per implementar R2M3 correctament.